

Project 1

Peter Williams

December 2025

1 Introduction

In this project, we compared the effect of different types and sizes of embeddings (pretrained using Glove, or random using Pytorch) on a relation connectives sense classifier for three different model architectures.

2 Methods

I preprocessed the data from the Penn Discourse Tree Bank such that the relations were organized into a Relation class that had the first argument, second argument, connective, and the sense of the connective in the class. Then for the actual representation of each relation for the model, I processed them into a string that began with the connective and then the first and second arguments, sequentially. The sense is the label. For simplicity, the senses are all labeled at level 1, of 3 possible levels of specificity, where 1 is the most broad level.

Each model is built using Pytorch. I built an MLP (allowing for different numbers of hidden layers), a logistic regression model, and a CNN using 1-dimensional convolutions. To embed the texts for the MLP and logistic regression model, which expects one embedding for the whole relation, I first padded/truncated the whole sequence, then embedded each token, and averaged the embedding of all the tokens into one sentence-level embedding. Then to boost the strength of the connective's embedding in the sentence-level representation, I added its embedding to the sentence-level again, acting as a sort of residual connection.

For the experiments, I first ran a grid search to find the best hyperparameters for an MLP, and then used those hyperparameters for the next experiments, which involved testing different parts of the architecture. In experiment one, I tested different numbers of hidden sizes and hidden layers on an MLP. In experiment two, I trained multiple MLPs using pre-trained and random embeddings with three different hidden sizes (50, 100, 200). In experiment three, I trained an MLP, CNN, and logistic regression model using Glove embeddings with the best size from experiment two to try to maximize their evaluation metrics. For each of these experiments, I used accuracy, F1, precision, and recall as evaluation metrics.

3 Results

Overall, the results were fairly poor. In some of my early testing, my models were performing at around 80% accuracy, and somewhere along the way, something changed or was lost and the best performances I could get when running the full set of experiments was around 40% on the CNN. Unfortunately, because I was still in early testing to make sure the pieces were all working, I wasn't saving the results from the earliest tests that achieved such performance. That's certainly something that I will change in the future, and make logging available and more consistent from the beginning of a project.

If I were able to spend more time on this project, I would be able to figure out what's going wrong, but I can't find anything. My main suspicion is something in the model architecture that I may have changed at some point, but I can't find anything now that looks wrong, or causes the model to not be able to run through a full training.

The results from each experiment are available in the `experiments` folder.

4 Analysis

One area that could have caused such poor performance is the way that everything was laid out. Since I performed each experiment in succession, using the best results from the previous experiment, I didn't actually end up testing every possible configuration. This approach makes sense to minimize the amount of time spent in a grid search by progressively narrowing down some of the possible configurations, but by only pulling the single best configuration from each prior experiment, it could be losing several other valid configurations that perform comparably and may perform better with the next experiment's variations.

Another thing that was interesting is that I noticed in my original grid search that some models were heavily overfitting, which obviously causes poor performance on the test set. Another option would be to make use of early stopping in order to get better results on those configurations, and see if they could outperform what I have.

5 Conclusion

Overall, I believe that all my experiments and the code that I ran were valid, but I must have been missing something that caused such poor performance. With more time, I would be able to explore more of the options and see what was contributing to the performance, and train models that were closer to the initial results I was achieving.